

Sesión 14 — Una app no es solo lógica: también es cómo se siente

Curso D4G (Diseña 4 Games) · Duración: 1 hora · UX y decisiones

*Tu app puede funcionar perfectamente por dentro y aun así ser mala. La diferencia entre «funciona» y «se puede usar» se llama **UX**. Diseñar UX es tomar decisiones sobre qué siente y entiende la persona que juega, no añadir más lógica.*

1 Idea central de la sesión

Un sistema correcto por dentro puede seguir siendo un mal producto si la persona que lo usa no entiende qué pasa. La **experiencia de usuario (UX)** es la capa que traduce el estado de tu app en algo comprensible. No es decoración: es **comunicación**.

2 Conexión con conocimientos previos

Ya tienes lo difícil resuelto. En sesiones anteriores construiste:

- Un **estado centralizado** (`estado`) que sabe en qué pantalla estás (`pantallaActual`), de quién es el turno y la puntuación.
- Una **navegación por estado** (`navegar()`) y un renderizado que obedece al estado (`actualizarUI()`).
- Tus dos juegos ya detectan victoria, empate, errores y movimientos inválidos.

En la Sesión 13 hiciste **testing**: pensaste «¿qué puede fallar?». Hoy damos el paso siguiente: «**cuando algo falla o cambia, ¿cómo se entera el usuario?**». El testing protege el sistema; la UX protege a la persona.

En Scratch, cuando ganabas, sonaba algo o un sprite decía «¡GANASTE!». Eso ya era UX, aunque no lo llamabas así. Hoy lo hacemos a propósito y con criterio.

3 Explicación clara del concepto

Tu app cambia de estado constantemente: turnos, errores, victorias, letras acertadas. El problema es que **el estado vive en JavaScript, pero el usuario no ve JavaScript**. Solo ve la pantalla. Una buena UX responde en cada momento a tres preguntas:

1. **¿Qué está pasando ahora?** (¿de quién es el turno? ¿cuántos fallos llevo?)
2. **¿Qué acabo de hacer?** (¿mi clic contó? ¿esa letra ya la había usado?)
3. **¿Qué puedo hacer ahora?** (¿juego otra vez? ¿vuelvo al menú?)

Lo importante: **la UX no cambia tu lógica**. La función que detecta victoria es la misma. Lo que cambia es cómo el estado se traduce en mensajes y feedback dentro de `actualizarUI()`.

4 Diseño antes de código

Antes de tocar nada, decidimos qué responsabilidad tiene cada parte. Esto es lo más importante de la sesión:

Parte del sistema	Su trabajo en UX	Lo que NO debe hacer
Lógica de juego	Decidir <i>qué pasó</i> (victoria, error, empate) y guardarlo en el estado	Escribir mensajes ni tocar la pantalla
<code>estado</code>	Recordar la situación, incluida info para mensajes (ej: <code>estado.mensaje</code>)	Decidir colores o textos finales
<code>actualizarUI()</code>	Leer el estado y traducirlo a mensajes, colores y botones claros	Cambiar el estado

El mensaje que ve el usuario es una **consecuencia del estado**, no una decisión suelta. Encaja en el patrón de siempre:

evento → función → estado cambia → UI se actualiza (ahora con buen mensaje)

La UX no es un sistema nuevo: es darle un trabajo extra a `actualizarUI()` — no solo mostrar la pantalla correcta, sino mostrarla de forma **comprensible**.

5 Ejemplo aplicado al proyecto

En el Ahorcado, el usuario pulsa una letra que **ya había usado**.

✗ **Sin criterio de UX:** el sistema es correcto pero frustra, el usuario no entiende por qué su clic no hace nada.

```
function elegirLetra(letra) {
  if (estado.letrasUsadas.includes(letra)) {
    return // no pasa nada... el usuario no sabe por qué
  }
  // ...
}
```

✓ **Con UX:** la lógica guarda *qué pasó* en el estado, y la UI lo comunica.

```
function elegirLetra(letra) {
  if (estado.letrasUsadas.includes(letra)) {
    estado.mensaje = "Esa letra ya la has usado, prueba otra"
    actualizarUI()
    return
  }
  // ... lógica normal
}
```

La lógica decidió *qué pasó*; la UI decidió *cómo se cuenta*. Cada uno en su sitio. Lo mismo en Tic Tac Toe: al ganar, la UI debe decir «¡Gana X!» y ofrecer un botón claro de «Jugar otra vez».

6 Uso guiado de agentes de IA

Qué pedirle a la IA:

- «Revisa el flujo del Ahorcado y dime en qué momentos el usuario podría no entender qué ha pasado.»
- «Dame tres formas distintas de mostrar de quién es el turno en el Tic Tac Toe.»
- «¿Qué mensajes de error son confusos y cómo los harías más claros para alguien que nunca ha jugado?»

Qué revisar obligatoriamente: la IA tenderá a meter mensajes dentro de la lógica de juego. Comprueba que el texto se guarde en `estado` y se muestre en `actualizarUI()`, no escrito a lo bruto en medio de las reglas.

Dónde se equivoca: escribe mensajes que *suenan* profesionales pero son genéricos («Error 400», «Acción no válida»). Un buen mensaje le habla a tu usuario real: una persona jugando, no un programador.

La IA te ayuda a redactar mensajes y detectar huecos, pero eres tú quien decide qué siente el usuario. La IA no diseña la experiencia.

7 Preguntas de pensamiento crítico

1. Si tu juego funciona pero un amigo de 8 años no sabe de quién es el turno, ¿está terminado el juego? ¿Por qué?
2. ¿Por qué el mensaje debería guardarse en `estado.mensaje` y no escribirse directamente desde la lógica? ¿Qué problema evitamos?
3. ¿Cuál es la diferencia entre un error que **rompe** la app y uno que solo **confunde**? ¿Cuál es más peligroso para un producto?
4. ¿Cuándo es mejor *impedir* una acción inválida y cuándo es mejor *permitirla y avisar*?

8 Mini-reto para 1 hora

Reto: convierte un momento confuso en un momento claro.

1. **Elige UN momento** de cualquiera de tus dos juegos donde el usuario podría perderse (letra repetida, fin de partida, clic en casilla ocupada...).
2. **Diseña primero en papel** (sin código): ¿qué debería ver? ¿qué mensaje? ¿qué botón? ¿qué color?
3. **Decide la responsabilidad:** ¿qué guarda la lógica en `estado`? ¿qué muestra `actualizarUI()`?
4. **Implementa** usando `estado.mensaje` y mostrándolo desde `actualizarUI()`.
5. **Prueba con alguien** que no haya visto tu juego. ¿Lo entiende sin que le expliques nada?

Criterio de éxito: no es «tiene más código». Es «ahora se entiende sin explicación».

9 Qué NO hacer

- ✗ Escribir mensajes directamente desde la lógica de juego (rompe la separación de responsabilidades).
- ✗ Cambiar la UI sin pasar por el estado (antipatrón ya conocido: la UI obedece, no decide).
- ✗ Usar mensajes genéricos de programador («Invalid input») en vez de hablarle al jugador.
- ✗ Confundir UX con «ponerlo bonito». Primero claridad, después estética.
- ✗ Añadir librerías o pop-ups complejos para algo que tu estado ya puede resolver.
- ✗ Pedirle a la IA «haz que la UX sea buena» sin haber decidido tú primero qué tiene que entender el usuario.

■ CIERRE DE LA SESIÓN

«Un sistema correcto que confunde al usuario sigue siendo un sistema roto.»

La UX no es magia ni decoración: es tu estado, bien comunicado.

Patrón central del curso: evento → función → estado cambia → UI se actualiza · Stack: HTML + CSS + JavaScript puro · Regla de oro: «Sal con una idea clara, no con más código.»